Module 7 Lab: Session 1: Contracts That Survive Change

Module	M7: Advanced Web API Development
Exercises	1 (API Versioning + Deprecation), 2 (CQRS, MediatR Pipelines, `Result<T>`, IExceptionHandler)
What you build	A TMS API that serves V1 and V2 simultaneously, signals V1's sunset in HTTP headers, and routes every business decision through MediatR with typed `Result<T,E>` and a global RFC 7807 `ProblemDetails` translator

Welcome to the Hardening Sprint

In M6 you built the API contract, clean resource URLs, DTOs, status codes, pagination, Scalar docs. Hand that to the Angular team in M8 and they can integrate.

Then production happens.

Wednesday, 09:14. A teammate pushes a change to the V1 course endpoint that renames a JSON field. Fifty tablets in a rural training centre running last quarter's mobile app stop working at 09:00. The dashboard team's deploy is rolled back. The rural centre is told to wait. Two days of trust evaporate.

Friday, 16:30. The team opens `EnrollmentsController.cs` to add a small audit-log line. They scroll past 400 lines of validation, conditional logic, email triggers, and database calls before finding the controller action. The "small" change ships with a typo because no one can hold the whole file in their head.

Both stories have the same root cause: the team wrote a CRUD API and called it done. **Session 1 is where you stop doing that.** By the end of today your TMS API can evolve without breaking existing clients (Exercise 1) and is structured so a single line of business logic can be added without re-reading the controller (Exercise 2).

> **Stop and tell the person next to you, in one sentence each:** what would have prevented Wednesday's incident, and what would have prevented Friday's incident? Hold those two sentences in your head. when you finish today, you will have shipped both fixes.

Before You Begin: Session 1 sync

You should already have completed M6 with a working TMS API exposing **`GET /api/courses`** (list and detail as you built in M6), **`POST /api/courses/{courseId}/enrollments`** (nested enrollment from M6), and Scalar at the

explorer path your TMS project mapped (typically **`https://localhost:5001/scalar/v1`**, or **`/scalar`** if that is what resolves in the browser). From the workspace root:

```
cd TmsApi
dotnet build
dotnet test
dotnet run --project TmsApi.Api
```

Confirm the API starts on `https://localhost:5001` (or the HTTPS URL shown in your `[launchSettings.json]`). **Stop here and fix any M6 build errors before continuing.** M7 piles new conventions on top. Stacking on a broken foundation wastes the whole session.

Install the Session 1 packages:

```
dotnet add TmsApi.Api package Asp.Versioning.Http
dotnet add TmsApi.Api package Asp.Versioning.Mvc.ApiExplorer
dotnet add TmsApi.Application package MediatR
dotnet add TmsApi.Application package FluentValidation
dotnet add TmsApi.Application package
FluentValidation.DependencyInjectionExtensions
```

Run `dotnet build` again after the installs. If it fails with `Asp.Versioning.Http` not found, you ran `dotnet add` against the wrong project, the versioning packages belong to the API project, the validation packages to the Application project. Fix it before moving on.

Exercise 1: API Versioning and Deprecation Strategy

Why this exists: Without versioning, every change is a breaking change. With URL-segment versioning *and* Sunset/Deprecation headers *and* a one-page policy, every change is either non-breaking (additive) or scheduled (sunset). That difference is the boundary between “we ship features” and “we keep getting hauled into incident calls.”

Step 1 Configure versioning in Program.cs

Open `TmsApi.Api/Program.cs`. Find the line that says `var builder = WebApplication.CreateBuilder(args);`. Below your existing service registrations (after `builder.Services.AddControllers()` if you have it; before `var app = builder.Build();`), add the versioning services:

```
using Asp.Versioning;            // add to the top of the file with the other
usings

builder.Services.AddApiVersioning(options =>
{
    options.DefaultApiVersion = new ApiVersion(1, 0);
    options.AssumeDefaultVersionWhenUnspecified = true;
    options.ReportApiVersions = true;
```

```
    options.ApiVersionReader = new UrlSegmentApiVersionReader();
})
.AddApiExplorer(options =>
{
    options.GroupNameFormat = "'v'VVV";
    options.SubstituteApiVersionInUrl = true;
});
```

What each option does, in plain language.

- `DefaultApiVersion`: what version a client gets if it does not ask for one.
- `AssumeDefaultVersionWhenUnspecified`: controls whether unversioned URLs (`/api/courses`) are accepted. Leave on while migrating; turn off once everyone is versioned.
- `ReportApiVersions`: adds `api-supported-versions: 1.0, 2.0` to every response. The browser tab and the curl output both surface what versions exist; partner teams find this far more useful than your wiki.
- `UrlSegmentApiVersionReader`: tells the framework versions live in the URL (`/api/v1/...`).
- `GroupNameFormat = "'v'VVV"` and `SubstituteApiVersionInUrl = true` make Scalar render `v1` and `v2` as separate documents.

Run `dotnet build`. It must compile. If it does not, the most common error is missing `using Asp.Versioning;` at the top.

Step 2 Create the V1 controller

V1 is the version your existing clients depend on. Treat it as a frozen contract: no field renames, no new required fields, no removed fields. Create the file `TmsApi.Api/Controllers/V1/CoursesController.cs`:

```
using Asp.Versioning;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace TmsApi.Api.Controllers.V1;

[ApiController]
[Route("api/v{version:apiVersion}/courses")]
[ApiVersion("1.0")]
public class CoursesController(TmsDbContext context) : ControllerBase
{
    [HttpGet]
    public async Task<IActionResult> GetCourses(
        [FromQuery] int page = 1,
        [FromQuery] int pageSize = 20,
        CancellationToken ct = default)
    {
```

```
        page = Math.Max(1, page);
        pageSize = Math.Clamp(pageSize, 1, 50);

        var baseQuery = context.Courses.AsNoTracking();

        var totalCount = await baseQuery.CountAsync(ct);

        var items = await baseQuery
            .OrderBy(c => c.Title)
            .Skip((page - 1) * pageSize)
            .Take(pageSize)
            .Select(c => new
            {
                c.Id,
                c.Code,
                c.Title,
                c.MaxCapacity,
                EnrollmentCount = c.Enrollments.Count
            })
            .ToListAsync(ct);

        var totalPages = (int)Math.Ceiling(totalCount / (double)pageSize);

        return Ok(new
        {
            items,
            totalCount,
            page,
            pageSize,
            totalPages,
            hasNext = page < totalPages,
            hasPrevious = page > 1
        });
    }
}
```

Two details that are not obvious. Put **V1** and **V2** course controllers in **different .NET namespaces** so they are two distinct controller types; **`[ApiVersion("1.0")]`** vs **`[ApiVersion("2.0")]`** plus the **`v{version:apiVersion}`** segment are what route a request to the right type. The `[Route("api/v{version:apiVersion}/courses")]` template uses the `apiVersion` route constraint registered by `AddApiVersioning`; do not invent your own template like a hard-coded `[Route("api/v1/courses")]` on the class only, then every new version needs a duplicate controller class for the same resource.

The list action preserves the **M6 TMS API contract** for catalogues: **`items`** plus paging metadata at the JSON root, and each row carries **`id`**, **`code`**, **`title`**, **`maxCapacity`**, **`enrollmentCount`** so tablets and Angular list screens stay on-spine.

Step 3 Create the V2 controller

V2 is where the dashboard team's enrichment lives: the **`data` / `meta` / `links`** envelope from the Module 7 spine (rows in **`data`**, paging fields in **`meta`**, discoverability hints in **`links`**). Create `TmsApi.Api/Controllers/V2/CoursesController.cs`:

```
using Asp.Versioning;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace TmsApi.Api.Controllers.V2;

[ApiController]
[Route("api/v{version:apiVersion}/courses")]
[ApiVersion("2.0")]
public class CoursesController(TmsDbContext context) : ControllerBase
{
    [HttpGet]
    public async Task<IActionResult> GetCourses(
        [FromQuery] int page = 1,
        [FromQuery] int pageSize = 20,
        CancellationToken ct = default)
    {
        page = Math.Max(1, page);
        pageSize = Math.Clamp(pageSize, 1, 50);

        var baseQuery = context.Courses.AsNoTracking();

        var totalCount = await baseQuery.CountAsync(ct);

        var rows = await baseQuery
            .OrderBy(c => c.Title)
            .Skip((page - 1) * pageSize)
            .Take(pageSize)
            .Select(c => new
            {
                c.Id,
                c.Title,
                c.Code,
                c.MaxCapacity,
                EnrollmentCount = c.Enrollments.Count
            })
            .ToListAsync(ct);

        var totalPages = (int)Math.Ceiling(totalCount / (double)pageSize);
        var hasNext = page < totalPages;
        var hasPrevious = page > 1;

        return Ok(new
```

```
        {
            data = rows,
            meta = new
            {
                totalCount,
                page,
                pageSize,
                totalPages,
                hasNext,
                hasPrevious
            },
            links = new
            {
                self = $"/api/v2/courses?page={page}&pageSize={pageSize}",
                next = hasNext
                    ? $"/api/v2/courses?page={page + 1}&pageSize={pageSize}"
                    : (string?)null,
                prev = hasPrevious
                    ? $"/api/v2/courses?page={page - 1}&pageSize={pageSize}"
                    : (string?)null,
                enroll = "/api/v2/enrollments"
            }
        });
    }
}
```

Run the API and prove both versions answer. In the terminal:

```
dotnet run --project TmsApi.Api
```

In a second terminal (leave the first showing logs):

```
curl -i https://localhost:5001/api/v1/courses
curl -i https://localhost:5001/api/v2/courses
```

You must see:

- `/api/v1/courses` returns **paged** JSON (root object with **`items`** + **`totalCount`** / **`page`**, same **TMS API contract** family as the programme pivot), **not** a bare `[...]` array at the root.
- `/api/v2/courses` returns the **envelope** from the Module 7 spine: **`data`** (the rows for that page), **`meta`**, and **`links`** (**`self`**, **`next`**, …).
- Both responses include the header `api-supported-versions: 1.0, 2.0`.

If you get `AmbiguousMatchException` at startup, both controllers are not in distinct namespaces, fix the namespaces and re-run. If V1 returns 404, the `v{version:apiVersion}` route template is wrong (or `SubstituteApiVersionInUrl = true` is missing).

Step 4 Mark V1 as deprecated *with the right HTTP headers*

URL-segment versioning is the *mechanism*. The *discipline* is telling V1 clients the version is going away, and where to go next. Real APIs (Stripe, GitHub, Twilio) do this through three response headers on every V1 response:

- `Deprecation: true` V1 is officially deprecated (IETF draft).
- `Sunset: <RFC 7231 date>` the date V1 will stop responding (RFC 8594).
- `Link: </api/v2/courses>; rel="successor-version"` where the client should migrate (RFC 5988).

Add a small middleware that stamps V1 responses with these headers. Create `TmsApi.Api/Middleware/V1DeprecationMiddleware.cs`:

```
namespace TmsApi.Api.Middleware;

public class V1DeprecationMiddleware(RequestDelegate next)
{
    private static readonly DateTimeOffset SunsetDate =
        new(2026, 12, 31, 0, 0, 0, TimeSpan.Zero);

    public async Task InvokeAsync(HttpContext context)
    {
        context.Response.OnStarting(() =>
        {
            if (context.Request.Path.StartsWithSegments("/api/v1"))
            {
                context.Response.Headers["Deprecation"] = "true";
                context.Response.Headers["Sunset"] =
SunsetDate.ToString("R");
                context.Response.Headers["Link"] =

$"<{context.Request.Scheme}://{context.Request.Host}/api/v2{context.Request.P
ath.Value?[7..]}>; rel=\"successor-version\"";
            }
            return Task.CompletedTask;
        });

        await next(context);
    }
}
```

Wire it in `Program.cs` **before** `app.MapControllers()`:

```
app.UseMiddleware<V1DeprecationMiddleware>();
```

> **Why a middleware and not an attribute?** The middleware applies to every V1 endpoint without anyone remembering to decorate it. New V1 routes inherit

the deprecation policy automatically. Forgetting one route is exactly the kind of silent drift that breaks production migrations.

Why `Response.OnStarting` and not just setting headers directly? Inside `InvokeAsync(...)` you do not yet know what status code or path the controller will return. `OnStarting` is invoked once headers are flushed but before the body, the latest possible point that headers can still be added. Setting headers earlier is harmless here but bites you later when conditional headers depend on response state (you will see this pattern again with rate limiting in Session 2).

Prove the headers land. Restart the API. Then:

```
curl -i https://localhost:5001/api/v1/courses
```

Read the response headers carefully. You must see:

```
HTTP/1.1 200 OK
Content-Type: application/json
Deprecation: true
Sunset: Thu, 31 Dec 2026 00:00:00 GMT
Link: <https://localhost:5001/api/v2/courses>; rel="successor-version"
api-supported-versions: 1.0, 2.0
```

Then:

```
curl -i https://localhost:5001/api/v2/courses
```

The V2 response must **not** include `Deprecation`, `Sunset`, or `Link` headers. If V2 has them, the middleware path check is matching `/api/v2` too. fix the `StartsWithSegments("/api/v1")` line.

Stop and observe. Look at your terminal three lines of HTTP machinery just communicated, in the protocol's own words, that V1 is on a sunset clock and V2 is the destination. No email. No support page. No phone call. Every well-behaved client now knows the future of the API the moment it makes a request. **This is the senior-grade detail interviewers probe for** when they ask "how do you migrate clients off old API versions?"

Step 5 Write a one-page versioning policy

Create `docs/api-versioning-policy.md` in your repo. Real engineering teams have one, interviewers ask to see it. Write it in your own words, on one page, and cover:

1. **What counts as a breaking change:** removing a field, renaming a field, changing a status code, tightening validation, changing a default sort order.
2. **What counts as additive (non-breaking):** adding a new optional field, adding a new endpoint, adding a new optional query parameter.

3. **Sunset window:** how long V1 keeps running after V2 ships. The TMS commits to **6 months minimum** so rural training centres on quarterly maintenance schedules can migrate.
4. **Communication:** `Deprecation` / `Sunset` / `Link` headers from day one of V2; a CHANGELOG entry; an email to every team that holds an API key; a calendar invite for the V1 shutdown date.
5. **Skipping versions:** V1 → V3 is allowed; clients are not forced to migrate through every intermediate version.

Keep it short. One page, no decoration. **The audit test:** hand the file to your bench partner. They should be able to read it in three minutes and immediately know whether a proposed change is breaking. If they cannot, rewrite it.

> **Why this matters at interview.** The most common follow-up question to "how do you version your API?" is "how do you decide what is and is not a breaking change?" Without a written policy, the answer is opinion. With one, the answer is a one-page document committed in the repo. Senior engineers ship policy; juniors ship code. This is the moment you start shipping policy.

Step 6 (Optional, 15 min) Add a header-based reader as an escape hatch

Some partners (especially mobile clients with cached CDN URLs) prefer not to have version in the URL. ASP.NET supports multiple readers at once. In `AddApiVersioning`:

```
options.ApiVersionReader = ApiVersionReader.Combine(
    new UrlSegmentApiVersionReader(),
    new HeaderApiVersionReader("X-Api-Version"));
```

Now `GET /api/courses` with the header `X-Api-Version: 2.0` resolves to V2 even though the URL is unversioned. Document this in the policy doc as **partner-by-partner opt-in**, not the default, one of the two has to be primary, and URL-segment is more visible during incident response.

Exercise 1 Final evidence (do not skip)

Your repo must contain, all simultaneously:

- ☐ `TmsApi.Api/Controllers/V1/CoursesController.cs` returning the **TMS API contract** V1 paged list (**`items`** + paging fields).
- ☐ `TmsApi.Api/Controllers/V2/CoursesController.cs` returning the wrapped **`{ data, meta, links }`** envelope.
- ☐ `TmsApi.Api/Middleware/V1DeprecationMiddleware.cs` stamping the three headers on every V1 response.
- ☐ `docs/api-versioning-policy.md` one page, written in your own voice.
- ☐ A captured `curl -i` against `/api/v1/courses` showing the deprecation headers (paste the output into your repo as `docs/evidence/v1-deprecation-curl.txt` for the capstone).

- ☐ A captured `curl -i` against `/api/v2/courses` showing **no** deprecation headers.

Troubleshooting Exercise 1

Symptom	Likely cause	Fix
`AmbiguousMatchException` at startup	Two `CoursesController` classes in the same namespace	Move V1 to `TmsApi.Api.Controllers.V1` and V2 to `TmsApi.Api.Controllers.V2`
404 on `/api/v1/courses`	Route template missing `v{version:apiVersion}`	Use `[Route("api/v{version:apiVersion}/courses")]` exactly
Only one version shows in Scalar	`AddApiExplorer` not chained after `AddApiVersioning`	Chain it: `builder.Services.AddApiVersioning(...).AddApiExplorer(...)`
`Sunset` header missing on V1	`UseMiddleware` registered after `MapControllers`	Move `app.UseMiddleware<V1DeprecationMiddleware>()` *before* `app.MapControllers()`
`Sunset` header appears on V2 too	Path check is too loose	Use `context.Request.Path.StartsWithSegments("/api/v1")` not `Contains("v1")`
`Link` header is malformed	The `[7..]` slice cuts off the slash	Verify the path starts with exactly `/api/v1` (no trailing slash); log it once to confirm

Exercise 2: CQRS, MediatR Pipelines, `Result<T,E>`, and `IExceptionHandler`

Why this exists: the controller you ship to the Angular team is the contract. If that controller has business logic in it, every contract change forces a code change and every code change risks breaking the contract. CQRS keeps the contract layer thin (HTTP in, HTTP out) and the decision layer testable (commands, queries, handlers). The two pipeline behaviors and the global `IExceptionHandler` are what turn this from theory into a production pattern.

M6 → M7 enrollment shape. In M6, enrollment is **`POST /api/courses/{courseId}/enrollments`** with `courseId` in the path. This exercise introduces a **versioned, flat write**: **`POST /api/v2/enrollments`** with **`studentId`** and **`courseCode`** in the JSON body: and keeps reads on **`GET /api/v2/enrollments/{studentId}/schedule`**. Before you test, **remove or disable the old nested enrollment POST** (or any duplicate enrollment endpoint) so only one write

route owns enrollment; otherwise you will fight ambiguous routes or “which contract did I just hit?” confusion.

Handler dependencies. `EnrollStudentHandler` and `GetStudentScheduleHandler` require **`ICourseRepository`** and **`IEnrollmentRepository`** with at least: `GetByCodeAsync` (return a `Course` that includes **`Enrollments`** for the capacity check), `ExistsAsync(studentId, courseCode)`, `AddAsync`, and `GetByStudentIdAsync` (**include `Course`** for the schedule query). If M6 only registered **services**, add EF-backed implementations of these interfaces and register them **`AddScoped`** before you expect `dotnet run` to succeed. Match method names to the handlers below; use your real `TmsDbContext` and **`Enrollment`** model (**`CourseId`** foreign key per M5/M6, not a free-standing `CourseCode` column unless your schema actually has one).

Step 1 Define a typed `Result<T,E>` and a domain error type

The first thing to fix is the return contract. `bool + int? + string?` is not a result. It is three optional fields glued together with hope. Real enterprise CQRS uses a typed `Result<T,E>` with an `Error` value object so callers cannot forget which case they are in.

Create `TmsApi.Application/Common/EnrollmentError.cs`:

```
namespace TmsApi.Application.Common;

public sealed record EnrollmentError(string Code, string Message)
{
    public static EnrollmentError CourseNotFound(string code) =>
        new("course_not_found", $"Course '{code}' was not found.");

    public static EnrollmentError CourseFull(string title, int capacity) =>
        new("course_full", $"Course '{title}' is full (capacity
{capacity}).");

    public static EnrollmentError AlreadyEnrolled(int studentId, string code)
=>
        new("already_enrolled", $"Student {studentId} is already enrolled in
{code}.");
}
```

Each error variant carries a stable **machine-readable `Code`** (used in the `ProblemDetails.type` URI) and a **human-readable `Message`**. The Code is the contract clients can write `if (error.Code === 'course_full')` without parsing the message.

Create `TmsApi.Application/Common/Result.cs`:

```
namespace TmsApi.Application.Common;

public readonly record struct Result<TValue, TError>
{
    private readonly TValue? _value;
```

```
    private readonly TError? _error;
    public bool IsSuccess { get; }

    private Result(TValue value) { _value = value; _error = default;
IsSuccess = true; }
    private Result(TError error) { _value = default; _error = error;
IsSuccess = false; }

    public static Result<TValue, TError> Success(TValue value) => new(value);
    public static Result<TValue, TError> Failure(TError error) => new(error);

    public TValue Value => IsSuccess
        ? _value!
        : throw new InvalidOperationException("Result is failure; call Match
instead of Value.");
    public TError Error => !IsSuccess
        ? _error!
        : throw new InvalidOperationException("Result is success; call Match
instead of Error.");

    public TOut Match<TOut>(Func<TValue, TOut> onSuccess, Func<TError, TOut>
onFailure) =>
        IsSuccess ? onSuccess(_value!) : onFailure(_error!);
}
```

> **Why not exceptions?** Exceptions are for *unexpected* failures, null pointers, dropped database connections, bugs. “Course is full” is an *expected business outcome* of a valid enrollment request. Throwing for an expected outcome makes stack traces noisy, hurts performance, and trains learners to treat the type system as advisory. **Use `Result<T,E>` for predictable domain failures; let exceptions be loud.**
>
> **Why `record struct`?** It is a value type, no heap allocation per result, and value equality is generated for free, which makes test assertions one line. The `readonly` modifier prevents handlers from mutating a result after creation, which is the kind of bug that takes an afternoon to chase.

Step 2 Define the command using `Result`

Create `TmsApi.Application/Enrollments/Commands/EnrollStudentCommand.cs`:

```
using MediatR;
using TmsApi.Application.Common;

namespace TmsApi.Application.Enrollments.Commands;

public record EnrollStudentCommand(int StudentId, string CourseCode)
    : IRequest<Result<EnrollmentCreated, EnrollmentError>>;
```

```
public record EnrollmentCreated(int EnrollmentId, int StudentId, string
CourseCode);
```

The command's return type `IRequest<Result<EnrollmentCreated, EnrollmentError>>` is the contract MediatR uses to find the right handler. Note that the controller does not see this type at all; it sees `IMediator.Send(command)` and unwraps the result. The strong typing protects the *handler*, not the controller.

Step 3 Create the handler

Create `TmsApi.Application/Enrollments/Commands/EnrollStudentHandler.cs`:

```
using MediatR;
using TmsApi.Application.Common;
using TmsApi.Application.Interfaces;
using TmsApi.Domain.Entities;

namespace TmsApi.Application.Enrollments.Commands;

public class EnrollStudentHandler(
    IEnrollmentRepository enrollmentRepo,
    ICourseRepository courseRepo)
    : IRequestHandler<EnrollStudentCommand, Result<EnrollmentCreated,
EnrollmentError>>
{
    public async Task<Result<EnrollmentCreated, EnrollmentError>> Handle(
        EnrollStudentCommand command, CancellationToken ct)
    {
        var course = await courseRepo.GetByCodeAsync(command.CourseCode, ct);
        if (course is null)
            return Result<EnrollmentCreated, EnrollmentError>.Failure(
                EnrollmentError.CourseNotFound(command.CourseCode));

        if (course.Enrollments.Count >= course.MaxCapacity)
            return Result<EnrollmentCreated, EnrollmentError>.Failure(
                EnrollmentError.CourseFull(course.Title,
course.MaxCapacity));

        if (await enrollmentRepo.ExistsAsync(command.StudentId,
command.CourseCode, ct))
            return Result<EnrollmentCreated, EnrollmentError>.Failure(
                EnrollmentError.AlreadyEnrolled(command.StudentId,
command.CourseCode));

        var enrollment = new Enrollment
        {
            StudentId = command.StudentId,
            CourseId = course.Id,
            EnrolledAt = DateTime.UtcNow
```

```
        };

        await enrollmentRepo.AddAsync(enrollment, ct);
        return Result<EnrollmentCreated, EnrollmentError>.Success(
            new EnrollmentCreated(enrollment.Id, enrollment.StudentId,
course.Code));
    }
}
```

> **Stop and read this handler carefully.** There is no `try/catch`. There is no `if/else` that throws. Every domain failure is an early return with a typed `Result.Failure(...)`. Every domain success is a single return at the bottom with `Result.Success(...)`. The handler is short, linear, and *exhaustively reviewable*. You can convince yourself, in 30 seconds, that every code path produces a typed result. This is the shape every CQRS handler should have.

Step 4 Create a query (commands write, queries read)

The “Q” in CQRS is “Query”: a *read* request. Queries are simpler than commands because they have no failure modes worth modelling (no “course full” on a read), so they return raw DTOs.

Create `TmsApi.Application/Enrollments/Queries/GetStudentScheduleQuery.cs`:

```
using MediatR;

namespace TmsApi.Application.Enrollments.Queries;

public record GetStudentScheduleQuery(int StudentId) : IRequest<ScheduleDto>;

public record ScheduleDto(int StudentId, List<ScheduleItemDto> Courses);
public record ScheduleItemDto(string CourseCode, string Title, string
Schedule);
```

Create `TmsApi.Application/Enrollments/Queries/GetStudentScheduleHandler.cs`:

```
using MediatR;
using TmsApi.Application.Interfaces;

namespace TmsApi.Application.Enrollments.Queries;

public class GetStudentScheduleHandler(IEnrollmentRepository repo)
    : IRequestHandler<GetStudentScheduleQuery, ScheduleDto>
{
    public async Task<ScheduleDto> Handle(
        GetStudentScheduleQuery query, CancellationToken ct)
    {
        var enrollments = await repo.GetByStudentIdAsync(query.StudentId,
ct);
```

```
        var items = enrollments.Select(e => new ScheduleItemDto(
            e.Course.Code,
            e.Course.Title,
            "TBD")).ToList();

        return new ScheduleDto(query.StudentId, items);
    }
}
```

The M6 **`Course`** shape does not include a separate timetable string; the third field is a **placeholder** until you add something like `Course.MeetingSummary` in your own schema. The teaching point is the query + DTO projection, not the column name.

Step 5 Create a `ValidationBehavior` pipeline

Behaviors run *before* the handler. They are how you keep cross-cutting concerns (validation, logging, auth, metrics) out of every handler. Create `TmsApi.Application/Behaviors/ValidationBehavior.cs`:

```
using FluentValidation;
using MediatR;

namespace TmsApi.Application.Behaviors;

public class ValidationBehavior<TRequest, TResponse>(
    IEnumerable<IValidator<TRequest>> validators)
    : IPipelineBehavior<TRequest, TResponse> where TRequest : notnull
{
    public async Task<TResponse> Handle(
        TRequest request,
        RequestHandlerDelegate<TResponse> next,
        CancellationToken ct)
    {
        if (!validators.Any())
            return await next();

        var context = new ValidationContext<TRequest>(request);
        var failures = validators
            .Select(v => v.Validate(context))
            .SelectMany(result => result.Errors)
            .Where(f => f is not null)
            .ToList();

        if (failures.Count > 0)
            throw new ValidationException(failures);

        return await next();
```

```
    }
}
```

Create `TmsApi.Application/Enrollments/Commands/EnrollStudentValidator.cs`:

```
using FluentValidation;

namespace TmsApi.Application.Enrollments.Commands;

public class EnrollStudentValidator : AbstractValidator<EnrollStudentCommand>
{
    public EnrollStudentValidator()
    {
        RuleFor(x => x.StudentId).GreaterThan(0)
            .WithMessage("Student ID must be a positive number.");
        RuleFor(x => x.CourseCode).NotEmpty()
            .WithMessage("Course code is required.");
        RuleFor(x => x.CourseCode).Matches(@"^[A-Z]{3}-\d{3}$")
            .WithMessage("Course code must follow the format XXX-000 (e.g.,
CSE-101).");
    }
}
```

> **Why `throw` here, after we just said "do not throw"?** Validation failures are a programming-or-input failure, not a business outcome. The request is *malformed*. Rejecting it with `400 Bad Request` is one line in `IExceptionHandler` (Step 7). If we returned `Result.Failure(...)` from validation, every caller would have to translate it back to `400` duplicated logic in every controller. The exception is the cheap, central path.

Step 6 Add a `LoggingBehavior` with correlation IDs

A real CQRS pipeline has more than one behavior. The next one every team adds is structured logging with a correlation id, so a request can be traced from the controller through the handler to the database. Create `TmsApi.Application/Behaviors/LoggingBehavior.cs`:

```
using System.Diagnostics;
using MediatR;
using Microsoft.Extensions.Logging;

namespace TmsApi.Application.Behaviors;

public class LoggingBehavior<TRequest, TResponse>(
    ILogger<LoggingBehavior<TRequest, TResponse>> logger)
    : IPipelineBehavior<TRequest, TResponse> where TRequest : notnull
{
    public async Task<TResponse> Handle(
        TRequest request,
```

```
        RequestHandlerDelegate<TResponse> next,
        CancellationToken ct)
    {
        var requestName = typeof(TRequest).Name;
        var correlationId = Activity.Current?.TraceId.ToString() ??
Guid.NewGuid().ToString("N");
        var stopwatch = Stopwatch.StartNew();

        using var scope = logger.BeginScope(new Dictionary<string, object>
        {
            ["RequestName"] = requestName,
            ["CorrelationId"] = correlationId
        });

        logger.LogInformation("Handling {RequestName} (cid={CorrelationId})",
requestName, correlationId);

        try
        {
            var response = await next();
            stopwatch.Stop();
            logger.LogInformation(
                "Handled {RequestName} in {ElapsedMs}ms
(cid={CorrelationId})",
                requestName, stopwatch.ElapsedMilliseconds, correlationId);
            return response;
        }
        catch (Exception ex)
        {
            stopwatch.Stop();
            logger.LogError(ex,
                "Failed {RequestName} after {ElapsedMs}ms
(cid={CorrelationId})",
                requestName, stopwatch.ElapsedMilliseconds, correlationId);
            throw;
        }
    }
}
```

Order matters and the wrong order is a silent failure. When multiple behaviors are registered, MediatR runs them in **registration order**. Register `LoggingBehavior` first so it wraps `ValidationBehavior`. That way the log scope is open before validation throws, and you see a `Failed EnrollStudentCommand after 3ms` line for every validation rejection. Reverse the order and a validation rejection appears as silent dead air in the logs while a 400 goes back to the client. You will spend an afternoon debugging “why is my logging not working” before you find this.

Step 7 Wire up `IExceptionHandler` for global `ProblemDetails`

`ValidationBehavior` throws `ValidationException`. Other unexpected things (`NullReferenceException`, EF `DbUpdateException`) can throw too. None of those should reach the controller unhandled. .NET 10 ships `IExceptionHandler` for exactly this one place to translate exceptions into RFC 7807 `ProblemDetails`. Create `TmsApi.Api/ExceptionHandlers/GlobalExceptionHandler.cs`:

```
using FluentValidation;
using Microsoft.AspNetCore.Diagnostics;
using Microsoft.AspNetCore.Mvc;

namespace TmsApi.Api.ExceptionHandlers;

public class GlobalExceptionHandler(ILogger<GlobalExceptionHandler> logger) :
IExceptionHandler
{
    public async ValueTask<bool> TryHandleAsync(
        HttpContext httpContext, Exception exception, CancellationToken ct)
    {
        var (status, title, detail, errors) = exception switch
        {
            ValidationException ve => (
                StatusCodes.Status400BadRequest,
                "Validation failed",
                "One or more fields are invalid. See errors for details.",
                (IDictionary<string, string[]>?)ve.Errors
                    .GroupBy(e => e.PropertyName)
                    .ToDictionary(g => g.Key, g => g.Select(e =>
e.ErrorMessage).ToArray())),
            _ => (
                StatusCodes.Status500InternalServerError,
                "Server error",
                $"An unexpected error occurred. Trace ID:
{httpContext.TraceIdentifier}",
                null)
        };

        if (status == StatusCodes.Status500InternalServerError)
            logger.LogError(exception, "Unhandled exception
(trace={TraceId})", httpContext.TraceIdentifier);

        var problem = new ProblemDetails
        {
            Status = status,
            Title = title,
            Detail = detail,
            Instance = httpContext.Request.Path
        };
```

```
        if (errors is not null) problem.Extensions["errors"] = errors;

        httpContext.Response.StatusCode = status;
        httpContext.Response.ContentType = "application/problem+json";
        await httpContext.Response.WriteAsJsonAsync(problem, ct);
        return true;
    }
}
```

The handler does three things every senior reviewer expects:

1. Maps `FluentValidation.ValidationException` to HTTP **400** with a structured `errors` dictionary keyed by field, the same shape ASP.NET produces for `[ApiController]` model state failures, so the Angular team has one error contract to handle for every kind of validation problem.
2. Logs unexpected exceptions with the request trace id, so support can find the failure in logs from the user's complaint email.
3. Never leaks an exception message or stack trace to the client.

Step 8 Register MediatR, behaviors, and the exception handler

Open `TmsApi.Api/Program.cs` and add (top of `builder.Services`):

```
builder.Services.AddMediatR(cfg =>
    cfg.RegisterServicesFromAssembly(typeof(EnrollStudentHandler).Assembly));

builder.Services.AddValidatorsFromAssembly(typeof(EnrollStudentValidator).Ass
embly);

// LoggingBehavior FIRST—it must wrap ValidationBehavior
builder.Services.AddTransient(typeof(IPipelineBehavior<,>),
typeof(LoggingBehavior<,>));
builder.Services.AddTransient(typeof(IPipelineBehavior<,>),
typeof(ValidationBehavior<,>));

builder.Services.AddExceptionHandler<GlobalExceptionHandler>();
builder.Services.AddProblemDetails();
```

After `var app = builder.Build();`, add `app.UseExceptionHandler()` **before** `app.UseRouting()` (or `app.MapControllers()` if you do not use `UseRouting`):

```
app.UseExceptionHandler();
```

> The order in `Program.cs` matters: `LoggingBehavior` is registered *before* `ValidationBehavior` so it runs *first* and wraps validation failures inside its log scope. If you register them the other way around, validation rejections will not appear in the structured log scope and you will think your logging is broken.

Step 9 Refactor the controller

Replace the business logic in your enrollments controller with MediatR dispatch and use the typed `Result` to translate domain errors into the right HTTP status:

```
using Asp.Versioning;
using MediatR;
using Microsoft.AspNetCore.Mvc;
using TmsApi.Application.Enrollments.Commands;
using TmsApi.Application.Enrollments.Queries;

[ApiController]
[Route("api/v{version:apiVersion}/enrollments")]
[ApiVersion("2.0")]
public class EnrollmentsController(IMediator mediator) : ControllerBase
{
    [HttpPost]
    public async Task<IActionResult> Enroll(
        EnrollStudentCommand command, CancellationToken ct)
    {
        var result = await mediator.Send(command, ct);
        return result.Match<IActionResult>(
            onSuccess: created => CreatedAtAction(
                nameof(GetSchedule),
                new { studentId = created.StudentId },
                created),
            onFailure: error =>
            {
                var status = error.Code switch
                {
                    "course_not_found" => StatusCodes.Status404NotFound,
                    "course_full" or "already_enrolled" =>
StatusCodes.Status409Conflict,
                    _ => StatusCodes.Status400BadRequest
                };
                return Problem(
                    statusCode: status,
                    title: "Enrollment rejected",
                    detail: error.Message,
                    type: $"https://tms.local/errors/{error.Code}");
            });
    }

    [HttpGet("{studentId}/schedule")]
    public async Task<IActionResult> GetSchedule(
        int studentId, CancellationToken ct)
    {
        var schedule = await mediator.Send(
            new GetStudentScheduleQuery(studentId), ct);
        return Ok(schedule);
```

```
    }
}
```

The mapping `course_not_found → 404` and `course_full / already_enrolled → 409` is REST-correct. A `400` says "your request is malformed", that is what `IExceptionHandler` already returns for validation. A `404` says "your request is well-formed but the resource you named does not exist." A `409 Conflict` says "your request is well-formed and the resource exists, but the resource state forbids this operation." Keep those three error categories distinct; lazy controllers throw everything as `400` and lose the diagnostic value of the status code.

Step 10 Test the entire pipeline end-to-end (this is the audit moment)

Run the API:

```
dotnet build
dotnet run --project TmsApi.Api
```

In a second terminal, run all four cases. **Watch the API's console log alongside each curl** the audit is in both places.

Test 1: Valid enrollment.

```
curl -i -X POST https://localhost:5001/api/v2/enrollments \
  -H "Content-Type: application/json" \
  -d '{"studentId": 1, "courseCode": "CSE-101"}'
```

Expected response: `HTTP/1.1 201 Created` with body `{"enrollmentId":1,"studentId":1,"courseCode":"CSE-101"}` and a `Location` header pointing at `/api/v2/enrollments/1/schedule`.

Expected log lines (in order):

```
info: Handling EnrollStudentCommand (cid=…)
info: Handled EnrollStudentCommand in 47ms (cid=…)
```

Test 2: Invalid course-code format → 400 from `IExceptionHandler`.

```
curl -i -X POST https://localhost:5001/api/v2/enrollments \
  -H "Content-Type: application/json" \
  -d '{"studentId": 1, "courseCode": "invalid"}'
```

Expected response: `HTTP/1.1 400 Bad Request` with `Content-Type: application/problem+json` and a body containing `"errors": { "courseCode": ["Course code must follow the format XXX-000 (e.g., CSE-101)."] }`.

Expected log lines:

```
info: Handling EnrollStudentCommand (cid=abc…)
fail: Failed EnrollStudentCommand after 3ms (cid=abc…)
       FluentValidation.ValidationException: ...
```

> **Audit moment.** The `(cid=abc…)` value in both lines is the **same correlation id** (often the current `Activity` trace id when one is present, otherwise the fallback from `LoggingBehavior`). Copy it and grep your logs for that token, you get one record of the whole request. **This is the moment "structured logging" stops being a buzzword.**

Test 3: Non-existent course code → 404 from `Result.Failure`.

```
curl -i -X POST https://localhost:5001/api/v2/enrollments \
  -H "Content-Type: application/json" \
  -d '{"studentId": 1, "courseCode": "ZZZ-999"}'
```

Expected response: `HTTP/1.1 404 Not Found` with `Content-Type: application/problem+json` and a body containing `"type": "https://tms.local/errors/course_not_found"`.

> **Stop. Notice what just happened.** The handler did not throw. It returned `Result.Failure(EnrollmentError.CourseNotFound(...))`. The controller pattern-matched on the error code and returned `Problem(statusCode: 404, ...)`. **Two different code paths produced two different `application/problem+json` bodies, but the Angular team consumes them with the same code.** That is the architectural payoff of `Result<T,E>` + `IExceptionHandler` working together.

Test 4: Duplicate enrollment → 409 from `Result.Failure`.

```
# Run Test 1 again—same student, same course
curl -i -X POST https://localhost:5001/api/v2/enrollments \
  -H "Content-Type: application/json" \
  -d '{"studentId": 1, "courseCode": "CSE-101"}'
```

Expected response: `HTTP/1.1 409 Conflict` with body containing `"type": "https://tms.local/errors/already_enrolled"`.

Exercise 2 Final evidence (do not skip)

Your repo must contain:

- ☐ `TmsApi.Application/Common/Result.cs` and `EnrollmentError.cs`.
- ☐ `TmsApi.Application/Enrollments/Commands/EnrollStudentCommand.cs` + `EnrollStudentHandler.cs` + `EnrollStudentValidator.cs`.
- ☐ `TmsApi.Application/Enrollments/Queries/GetStudentScheduleQuery.cs` + handler.

- ☐ `TmsApi.Application/Behaviors/ValidationBehavior.cs` and `LoggingBehavior.cs`, both registered as `IPipelineBehavior<,>` in `Program.cs` (Logging first, Validation second).
- ☐ `TmsApi.Api/ExceptionHandlers/GlobalExceptionHandler.cs` registered via `AddExceptionHandler<T>` and `app.UseExceptionHandler()` *before* `MapControllers`.
- ☐ `EnrollmentsController` with **no** business `if`/`switch` only HTTP-status mapping from `Result.Match(...)`.
- ☐ All four curl tests above produce the expected status codes and bodies.

Troubleshooting Exercise 2

Symptom	Likely cause	Quick fix
`No service for type 'IMediator'`	`AddMediatR` not called or wrong assembly reference	`cfg.RegisterServicesFromAssembly(typeof(EnrollStudentHandler).Assembly)`
Validation does not run	Validator not registered, or `ValidationBehavior` registered with wrong type	Confirm `AddValidatorsFromAssembly(typeof(EnrollStudentValidator).Assembly)`
Validation rejection returns `500`	`app.UseExceptionHandler()` not called, or registered after `MapControllers`	Add it immediately after `app = builder.Build();`, before `MapControllers`
`LoggingBehavior` logs nothing for a failed command	`ValidationBehavior` registered before `LoggingBehavior`	Reverse the order—Logging first, so it wraps Validation
`Result.Value` throws on failure	Calling `.Value` instead of `.Match(...)`	Always `.Match(onSuccess, onFailure)` the type system makes the failure case unmissable
Handler not resolved	`IRequestHandler<T, R>` signature does not match the command exactly	Compare types literally generics with `Result<TValue, TError>` are easy to mistype
`404` returns plain JSON, not `application/problem+json`	`controller.Problem(...)` not used	The controller's `onFailure` branch must call `Problem(statusCode: …, type: …)`

Session Integration Challenge: End-to-end on the V2 enrollment

When both exercises are done, the integration is the proof. Issue this single curl from a terminal and confirm every layer below it cooperates:

```
curl -i -X POST https://localhost:5001/api/v2/enrollments \
  -H "Content-Type: application/json" \
  -d '{"studentId": 1, "courseCode": "invalid"}'
```

What you should see, **in order**, in the API's console log and the response:

1. The middleware did **not** stamp deprecation headers (the request is on V2, not V1).
2. The request reached `EnrollmentsController.Enroll` and was dispatched via `IMediator.Send`.
3. `LoggingBehavior` opened a scope with `RequestName = EnrollStudentCommand` and a `CorrelationId`. A `Handling EnrollStudentCommand (cid=…)` log line is visible in the console.
4. `ValidationBehavior` ran and **threw** `ValidationException` because `"invalid"` does not match `^[A-Z]{3}-\d{3}$`.
5. The exception was *not* caught in the handler. It propagated out of MediatR.
6. `app.UseExceptionHandler()` invoked `GlobalExceptionHandler`, which produced HTTP `400` with `application/problem+json`, a `ProblemDetails` body, and an `errors.courseCode` array containing the regex message.
7. `LoggingBehavior` logged a `Failed EnrollStudentCommand after Xms (cid=…)` line with the **same** correlation id as the `Handling` line.

If any one of those is missing, you have a wiring gap somewhere in Session 1. The *order* of those events is the architectural shape, once it lines up, you have the contract the rest of M7 builds on.

Now run the **deprecation cross-check** to prove versioning still works:

```
curl -i https://localhost:5001/api/v1/courses
curl -i https://localhost:5001/api/v2/courses
```

Confirm V1 carries the three deprecation headers and V2 does not. Capture both outputs into `docs/evidence/session1-curl.txt`. The capstone marker grades versioning by reading that file.

> **The senior moment.** Look at what you just built. A single client request (`POST /api/v2/enrollments` with a malformed body) flowed through versioning routing, controller dispatch, MediatR, two pipeline behaviors, a validator, an exception, and an exception handler and produced one greppable correlation id, one well-formed `application/problem+json` response, and one set of structured log lines. **This is the architectural seam every other module in M7 plugs into.** Keep that shape stable today; the rest of the week is about clipping new features (caching, rate limiting, real-time push, resilience, observability) onto it.

Session 1 Checkpoint

Tick all of these before you leave the room:

- ☐ `GET /api/v1/courses` returns the V1 shape **and** carries `Deprecation: true`, a `Sunset` date, and a `Link` to V2.
- ☐ `GET /api/v2/courses` returns the V2 shape and does **not** carry deprecation headers.
- ☐ `docs/api-versioning-policy.md` exists, is one page, and a peer can read it in three minutes.
- ☐ `EnrollmentsController` has zero `if`/`switch` statements that talk about business rules.
- ☐ A POST with an invalid course code returns 400 with `Content-Type: application/problem+json` and `ProblemDetails.errors.courseCode` populated by `IExceptionHandler`.
- ☐ A POST against a non-existent course returns **404** with `type: https://tms.local/errors/course_not_found`.
- ☐ A duplicate enrollment returns **409** with `type: https://tms.local/errors/already_enrolled`.
- ☐ Each request produces *two* structured log lines (Handling/Handled or Handling/Failed) sharing a single `CorrelationId`.
- ☐ `dotnet test` is green.
- ☐ The four curl commands and their outputs are committed in `docs/evidence/session1-curl.txt`.

If any box is unchecked, do not move to Session 2. The next three sessions all attach to the surface you finished today; gaps here cascade.

Common Errors: Session 1

Symptom	Likely cause	Quick fix
`AmbiguousMatchException` at startup	Two controllers with the same name in the same namespace	Move V1 to `TmsApi.Api.Controllers.V1` and V2 to `TmsApi.Api.Controllers.V2`
V1 response missing deprecation headers	`UseMiddleware<V1DeprecationMiddleware>()` registered too late	Move it before `app.MapControllers()`
Validation rejection returns `500`	`app.UseExceptionHandler()` not called, or registered after `MapControllers`	Add it immediately after `app = builder.Build();`, before `MapControllers`

Symptom	Likely cause	Quick fix
`LoggingBehavior` logs nothing for a failed command	`ValidationBehavior` registered before `LoggingBehavior`	Reverse the order—Logging first wraps Validation
`Result.Value` throws on failure	Calling `.Value` instead of `.Match(...)`	Always `.Match(onSuccess, onFailure)` the type system makes the failure case unmissable
Handler not resolved	`IRequestHandler<T, R>` signature does not match the command exactly	Compare types literally generics with `Result<TValue, TError>` are easy to mistype
404 returns plain JSON, not `application/problem+json`	The controller's failure branch returns `NotFound()` instead of `Problem(...)`	Use `Problem(statusCode: 404, type: ...)` in the `onFailure` arm of `Match`

Bridge to Session 2

You now have a clean, versioned API surface and a CQRS pipeline that turns business decisions into typed results. Session 2 adds the two operational concerns the dashboard team will hit on Monday morning: **caching that survives a 50-request burst with one DB query**, and **rate limiting that distinguishes anonymous, free, and paid callers**. Both attach to the surface you finished today without it, neither makes sense.

Read **Module 7 Lab Session 2** (handout for the next classroom session) before the Session 2 lecture if you want to follow along faster.